

18 — Sécurité : mots de passe, JWT & rôles

Comment marche la sécurité du projet (tel que codé), pour répondre à l'oral « comment gérez-vous la connexion et les droits ? ».

En une phrase

À l'inscription, le mot de passe est **haché** (jamais stocké en clair). À la connexion, on vérifie le mot de passe puis on renvoie un **jeton JWT** contenant l'id et le rôle. À chaque requête protégée, deux **middlewares** vérifient : « es-tu connecté ? » (token) et « as-tu le bon rôle ? ».

Deux notions à distinguer (le jury adore) : - **Authentication** = « qui es-tu ? » → le token JWT. - **Autorisation** = « as-tu le droit ? » → le rôle.

1) Le mot de passe : haché avec bcrypt

On ne stocke **jamais** le mot de passe en clair. Fichier `API/auth/mdp.go` :

```
func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), 10)
    return string(bytes), err
}
```

- `bcrypt` transforme le mot de passe en une empreinte illisible (le *hash*).
- Le `10` est le **coût** (nombre de tours) : plus c'est haut, plus c'est lent à casser.
- On ne peut **pas** « déchiffrer » un hash bcrypt. À la connexion, on **compare** :

```
// API/bdd/userReq.go (à la connexion)
err = bcrypt.CompareHashAndPassword([]byte(user.MotDePasse), []byte(motDePasse))
```

`CompareHashAndPassword` re-hache le mot de passe saisi et le compare au hash en base. S'ils correspondent → mot de passe bon.

Question type : « Et si votre base fuit ? » → Les mots de passe sont hachés (bcrypt, coût 10), donc inutilisables tels quels par un attaquant.

2) Le jeton JWT : créé à la connexion

Une fois le mot de passe validé, on génère un **JWT** (`API/auth/jwt.go`) :

```
type Claims struct {
    UserID int    `json:"user_id"`
    Role   string  `json:"role"`
    jwt.RegisteredClaims
}

func GenerateJWT(userID int, role string) (string, error) {
    claims := &Claims{
        UserID: userID,
        Role:   role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
        },
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(jwtKey)
}
```

- Le token contient l'**id**, le **rôle** et une **date d'expiration** (24 h).
- Il est **signé** avec une clé secrète (`jwtKey`) en **HS256**. Personne ne peut le fabriquer ou le modifier sans la clé.
- Le backend le renvoie à la connexion ; le front le stocke dans `localStorage` (`login.js` : `localStorage.setItem("userRole", ...) + le token`).

Le JWT est **auto-porteur** : le serveur n'a rien à stocker. Il lui suffit de vérifier la signature pour faire confiance au contenu.

3) Middleware n°1 — « es-tu connecté ? » (`VerifyTokenMiddleware`)

Un **middleware** est une fonction qui **enveloppe** un handler : elle fait un contrôle *avant*, et n'appelle le vrai code (`next`) que si tout est bon.

```

func VerifyTokenMiddleware(next http.HandlerFunc) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        authHeader := r.Header.Get("Authorization") // "Bearer xxx"
        parts := strings.Split(authHeader, " ")
        tokenString := parts[1]

        claims := &Claims{}
        token, err := jwt.ParseWithClaims(tokenString, claims, func(t *jwt.Token) (interface{}, error) {
            return jwtKey, nil // on vérifie la signature
        })
        if err != nil || !token.Valid {
            http.Error(w, "Token invalide", http.StatusUnauthorized) // 401 → stop
            return
        }

        // on range l'id et le rôle DANS la requête pour la suite
        ctx := context.WithValue(r.Context(), "userID", claims.UserID)
        ctx = context.WithValue(ctx, "userRole", claims.Role)
        r = r.WithContext(ctx)

        next(w, r) // tout est bon → on lance le vrai handler
    }
}

```

Points clés : - Il lit l'en-tête `Authorization: Bearer <token>` . - Il **vérifie la signature** avec `jwtKey` .
 Token absent/faux/expiré → **401 Unauthorized**. - Il **extraie le rôle** du token et le range dans le `context` de la requête, pour que le middleware suivant puisse le lire.

4) Middleware n°2 — « as-tu le bon rôle ? » (`VerifyRoleMiddleware`)

```

func VerifyRoleMiddleware(next http.HandlerFunc, rolesAutorises ...string) http.HandlerFunc {
    // il RÉUTILISE VerifyTokenMiddleware → "connecté ?" déjà vérifié
    return VerifyTokenMiddleware(func(w http.ResponseWriter, r *http.Request) {
        role, _ := r.Context().Value("userRole").(string) // rôle rangé juste avant

        autorise := false
        for _, roleOk := range rolesAutorises {
            if role == roleOk {
                autorise = true
                break
            }
        }

        if !autorise {

```

```

        http.Error(w, "Accès refusé", http.StatusForbidden) // 403 → stop
        return
    }

    next(w, r) // rôle OK → vrai handler
})
}

```

Points clés : - Il **s'appuie sur** `VerifyTokenMiddleware` (pas de duplication) : d'abord connecté, ensuite le rôle. - `rolesAutorises ...string` = **paramètre variadique** : on passe un OU plusieurs rôles. - La boucle compare le rôle du token à la liste. Aucun match → **403 Forbidden**.

5) Comment on s'en sert dans les routes

```

// route ouverte à tout utilisateur CONNECTÉ :
http.HandleFunc("GET /admin/annonces", auth.VerifyTokenMiddleware(admin.GetAnnonces))

// route réservée aux ADMINS :
http.HandleFunc("GET /admin/users", auth.VerifyRoleMiddleware(admin.GetAllUsers, "Administrateur")

// route pour Admin OU Salarié :
http.HandleFunc("PUT /admin/xxx", auth.VerifyRoleMiddleware(admin.Xxx, "Administrateur", "Salarie

```



Piège fréquent : « quand je fais un fetch, j'envoie le rôle ? »

»

NON. Quand tu fais un `fetch`, tu n'envoies **que le token** :

```
fetch(url, { headers: { Authorization: "Bearer " + monToken } });
```

Le rôle n'est **pas** envoyé à part : il est **déjà à l'intérieur du token** (mis là par `GenerateJWT` à la connexion, et **signé**). Le backend ne fait pas confiance à ce que dit le front : il **ouvre le token lui-même** et lit `claims.Role` dans `VerifyTokenMiddleware`.

Pourquoi c'est fait comme ça ? Si on envoyait le rôle à la main (ex. un en-tête `X-Role: Administrateur`), n'importe qui pourrait le changer dans la console (F12) et se faire passer pour un admin. Avec le JWT, impossible de tricher : si tu modifies le rôle dans le token, la **signature ne correspond plus** → `token.Valid` = faux → **401**. On ne peut pas re-signer sans la clé secrète du serveur.

La nuance sur `localStorage.getItem("userRole")` : oui, le front lit un `userRole` dans le `localStorage` (utilisé par `navRole.js`, `auth_guard.js`). Mais c'est **uniquement pour l'affichage** (thème, navigation, redirection). Ce `userRole` ne décide **jamais** des droits côté serveur. Si quelqu'un le bidouille en `"Administrateur"` dans la console, il verra peut-être un autre thème, mais **toutes les requêtes protégées renverront 401/403**, car le vrai rôle est dans le **token signé**, pas dans le `localStorage`.

À retenir en une phrase : on envoie le **token** ; le rôle est **dedans** (signé) ; le serveur le **lit lui-même**. Le `userRole` du `localStorage` n'est qu'un confort d'affichage, jamais une preuve de droits.

6) Et côté frontend ?

Le front a aussi une barrière (`Frontend/script/auth_guard.js`, fonction `checkSession`) : - Pas de token dans `localStorage` → redirection vers `/login`. - Rôle qui ne correspond pas à la page → redirection vers `403.html`.

⚠ **Attention à l'oral** : cette barrière front est du **confort** (elle évite d'afficher une page interdite). La **vraie** sécurité est côté **backend** (les middlewares), car le front peut être contourné.

Les 401 vs 403 (à connaître par cœur)

Code	Signification	Middleware
401 Unauthorized	« Je ne sais pas qui tu es » (pas de token / invalide / expiré)	<code>VerifyTokenMiddleware</code>
403 Forbidden	« Je sais qui tu es, mais tu n'as pas le droit » (mauvais rôle)	<code>VerifyRoleMiddleware</code>

Questions possibles à l'oral

« **Où sont stockés les mots de passe ?** » Hachés en base avec **bcrypt** (coût 10). Jamais en clair, jamais déchiffrables. À la connexion on compare avec `CompareHashAndPassword`.

« **Comment le serveur sait qui est connecté sans base de session ?** » Grâce au **JWT auto-porteur** : le token contient l'id + le rôle et est signé. Le serveur vérifie juste la signature avec sa clé

secrète.

« **Qu'est-ce qui empêche un utilisateur de modifier son rôle dans le token ?** » La **signature HS256**. S'il change « Utilisateur » en « Administrateur », la signature ne correspond plus → `token.Valid` est faux → 401.

« **Différence authentication / autorisation ?** » Authentication = prouver **qui** on est (le token). Autorisation = vérifier ce qu'on a le **droit** de faire (le rôle). Deux middlewares séparés.

« **Que se passe-t-il quand le token expire ?** » Au bout de 24 h, `token.Valid` devient faux → 401 → le front redirige vers `/login`.